Applying the Crank-Nicolson Method to Model Heat Conduction

Yone Lim
April 26, 2026
MA 303
Dr. Hanh

Introduction

Modeling heat conduction in objects involves solving partial differential equations, which are generally difficult, if not impossible. Most solutions to partial differential equations are computed numerically using methods like Finite Difference and Finite Element.

The purpose of this project was to build on the numerical methods learned in Chapter 2 of *Differential Equations and Boundary Value Problems: Computing and Modeling, 6e* (C.Henry Edwards, David E. Penney, and David Calvis) by extending it to solving partial differential equations. This project will focus on modeling heat equations on bounded rectangular regions using the Crank-Nicolson scheme.

Brief Overview of Finite Difference Method

Note: Most of the text below are direct snippets from Chapter 6.1-6.4 from *Applied Partial Differential Equations, 3rd Edition* by J. David Logan [3], and Part 5 from *Partial Differential Equations for Scientists and Engineers* by Stanley J. Farlow [2].

From Chapter 2, we learned about Euler's method to approximate solutions to differential equations. This involved rewriting the derivative y' as a Taylor approximation

$$y'(t) = \frac{y(t+k) - y(t)}{k} + O(h) \qquad (1)$$

where $O(h)$ is the error term.

Suppose we have the differential equation of the form

$$y'(t) = g(t, y)$$

Dropping the error term in (1) and substituting it into the differential equation above yields

$$\frac{y(t+k) - y(t)}{k} = g(t, y)$$

and then solving for $f(x+h)$ yields

$$y(t+k) = k \cdot g(t, y) + y(t)$$

This is the **forward difference approximation** of f, since we are moving "forward" in x to approximate our value.

We can also approximate second derivatives of a function f using Taylor's theorem. If $u(x)$ is a sufficiently smooth function, then Taylor's theorem implies

$$u(x+h) = u(x) + u'(x)h + \frac{1}{2}u''(x)h^2 + \frac{1}{3!}u'''(x)h^3 + \cdots + \frac{1}{n!}f^{(n)}(x)h^n + O(h^{n+1})$$

Ending this series at the 4th term yields

$$u(x+h) = u(x) + u'(x)h + \frac{1}{2}u''(x)h^2 + \frac{1}{3!}u'''(x)h^3 + O(h^4) \qquad (2)$$

To obtain an approximation for u_{xx}, we write the equation above for h replaced by $-h$ to obtain

$$u(x-h) = u(x) - u'(x)h + \frac{1}{2}u''(x)h^2 - \frac{1}{3!}u'''(x)h^3 + O(h^4)$$

Adding this expression to (2) and solving for u_{xx} gives the **centered difference appoximation**

$$u_{xx} = \frac{u(x-h) - 2u(x) + u(x+h)}{h^2} + O(h^2)$$

Applying difference approximations to partial derivatives

To extend the difference approximations we learned in Chapter 2 in MA 303 to partial derivatives, we express each partial derivative as the difference approximation with respect to its variable of differentiation. For our application, we will be focusing on the 2 dimensional heat equation

$$\frac{\partial u}{\partial t} = \alpha\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right)$$

To find our difference approximation of $u(x,y,t)$, we plug in the following, which are similar to what we derived previously (without the error terms):

$$u_t(x,y,t) = \frac{u(x,y,t+k) - u(x,y,t)}{k}$$

$$\begin{aligned} u_{xx}(x,y,t) = \lambda_1 &\left[\frac{u(x+h,y,t+k) - 2u(x,y,t+k) + u(x-h,y,t+k)}{h^2}\right] \\ &+ (1-\lambda_1)\left[\frac{u(x+h,y,t) - 2u(x,y,t) + u(x-h,y,t)}{h^2}\right] \end{aligned}$$

$$\begin{aligned} u_{yy}(x,y,t) = \lambda_2 &\left[\frac{u(x,y+d,t+k) - 2u(x,y,t+k) + u(x,y-d,t+k)}{d^2}\right] \\ &+ (1-\lambda_2)\left[\frac{u(x,y+d,t) - 2u(x,y,t) + u(x,y-d,t)}{d^2}\right] \end{aligned}$$

Where k is the time-step, h is the step size in x, and d is the step size in y. λ_1 and λ_2 are chosen numbers in the interval $[0, 1]$. Notice that the approximations for u_{xx} and u_{yy} are *weighted averages* of the central difference approximations to the derivative u_{xx} and u_{yy} at time values t and $t + k$. In other words, we added up the central difference of u_{xx} at t and $t + k$ and took the weighted average, which is determined by λ_1. Same for u_{yy}.

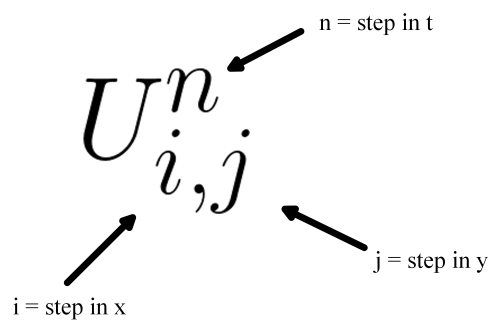
For the **Crank-Nicolson method**, $\lambda_1 = \lambda_2 = \frac{1}{2}$. It is the *ordinary average* of the two central differences we added together for u_{xx} and u_{yy}.

Applying Crank-Nicolson to Rectangle Region

To numerically model the 2D heat equation, the first step is to **discretize** the region of space-time where we want to obtain a solution. In this case, the region is a three-dimensional *mesh* of lattice points defined by

$$x_i = ih,\ y_j = jd,\ t_n = nk,\ i = 0, 1, \cdots, I;\ j = 0, 1, \cdots, J;\ n = 0, 1, \cdots, N$$

Where h, d, and k are spatial and temporal *step sizes* as in the previous section. Since the difference equations from the previous section are notationally dense, we follow a simpler notation:



Then our previous difference equations

$$u_t(x, y, t) = \frac{u(x, y, t + k) - u(x, y, t)}{k}$$

$$\begin{aligned} u_{xx}(x, y, t) = \lambda_1 &\left[\frac{u(x + h, y, t + k) - 2u(x, y, t + k) + u(x - h, y, t + k)}{h^2}\right] \\ &+ (1 - \lambda_1)\left[\frac{u(x + h, y, t) - 2u(x, y, t) + u(x - h, y, t)}{h^2}\right] \end{aligned}$$

$$\begin{aligned} u_{yy}(x, y, t) = \lambda_2 &\left[\frac{u(x, y + d, t + k) - 2u(x, y, t + k) + u(x, y - d, t + k)}{d^2}\right] \\ &+ (1 - \lambda_2)\left[\frac{u(x, y + d, t) - 2u(x, y, t) + u(x, y - d, t)}{d^2}\right] \end{aligned}$$

turns into their discretized forms

$$u_t(x_i, y_j, t_n) \approx \frac{U_{i,j}^{n+1} - U_{i,j}^n}{k}$$

$$u_{xx}(x_i, y_j, t_n) \approx \frac{1}{2}\left[\frac{U_{i+1,j}^{n+1} - 2U_{i,j}^{n+1} + U_{i-1,j}^{n+1}}{h^2} + \frac{U_{i+1,j}^n - 2U_{i,j}^n + U_{i-1,j}^n}{h^2}\right]$$

$$u_{yy}(x_i, y_i, t_n) \approx \frac{1}{2}\left[\frac{U_{i,j+1}^{n+1} - 2U_{i,j}^{n+1} + U_{i,j-1}^{n+1}}{d^2} + \frac{U_{i,j+1}^n - 2U_{i,j}^n + U_{i,j-1}^n}{d^2}\right]$$

Plugging these into the heat equation yields

$$\frac{\partial u}{\partial t} = \alpha\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right)$$

$$\begin{aligned}\implies \frac{U_{i,j}^{n+1} - U_{i,j}^n}{k} = \frac{\alpha}{2}\Bigg(&\frac{U_{i+1,j}^{n+1} - 2U_{i,j}^{n+1} + U_{i-1,j}^{n+1}}{h^2} + \frac{U_{i+1,j}^n - 2U_{i,j}^n + U_{i-1,j}^n}{h^2} \\ &+ \frac{U_{i,j+1}^{n+1} - 2U_{i,j}^{n+1} + U_{i,j-1}^{n+1}}{d^2} + \frac{U_{i,j+1}^n - 2U_{i,j}^n + U_{i,j-1}^n}{d^2}\Bigg)\end{aligned}$$

Multiplying both sides by k

$$\begin{aligned}U_{i,j}^{n+1} - U_{i,j}^n = \frac{k\alpha}{2}\Bigg(&\frac{U_{i+1,j}^{n+1} - 2U_{i,j}^{n+1} + U_{i-1,j}^{n+1}}{h^2} + \frac{U_{i+1,j}^n - 2U_{i,j}^n + U_{i-1,j}^n}{h^2} \\ &+ \frac{U_{i,j+1}^{n+1} - 2U_{i,j}^{n+1} + U_{i,j-1}^{n+1}}{d^2} + \frac{U_{i,j+1}^n - 2U_{i,j}^n + U_{i,j-1}^n}{d^2}\Bigg)\end{aligned}$$

and letting $r_x = \frac{k\alpha}{2h^2}$ and $r_y = \frac{k\alpha}{2d^2}$, we have

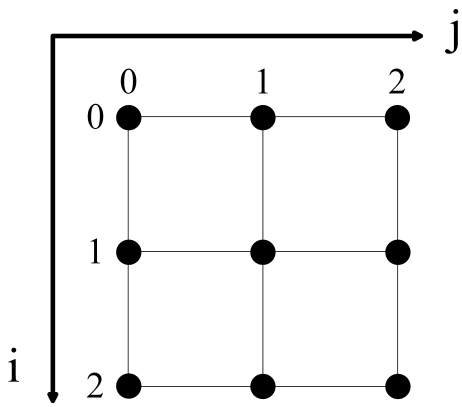
$$\begin{aligned}U_{i,j}^{n+1} - U_{i,j}^n = &\, r_x\left(U_{i+1,j}^{n+1} - 2U_{i,j}^{n+1} + U_{i-1,j}^{n+1} + U_{i+1,j}^n - 2U_{i,j}^n + U_{i-1,j}^n\right) \\ &+ r_y\left(U_{i,j+1}^{n+1} - 2U_{i,j}^{n+1} + U_{i,j-1}^{n+1} + U_{i,j+1}^n - 2U_{i,j}^n + U_{i,j-1}^n\right)\end{aligned}$$

Gathering all the "future terms " (the $n+1$ terms) to the left side, we have

$$\begin{aligned}&-r_x U_{i+1,j}^{n+1} - r_y U_{i,j+1}^{n+1} + (1 + 2r_x + 2r_y)U_{i,j}^{n+1} - r_x U_{i-1,j}^{n+1} - r_y U_{i,j-1}^{n+1} \\ &= r_x U_{i+1,j}^n + r_y U_{i,j+1}^n + (1 - 2r_x - 2r_y)U_{i,j}^n + r_x U_{i-1,j}^n + r_y U_{i,j-1}^n\end{aligned}$$

Using this relationship, we need to set up a tridiagonal system of equations such that $A\mathbf{u}^{n+1} = B\mathbf{u}^n$ [1]. That is, for **every** time step, we have to solve a system of equations for **all** the values at the lattive points.

To understand this, suppose a 3x3 discretized square lattice region ($i = 0, 1, 2 = I;\ j = 0, 1, 2 = J$)



and we want to compute the values of $\mathbf{u}^{n+1}$ ($t = n + 1$), where

$$\mathbf{u}^{n+1} = \begin{bmatrix} U_{0,0}^{n+1} \\ U_{0,1}^{n+1} \\ U_{0,2}^{n+1} \\ \vdots \\ U_{i,j}^{n+1} \\ \vdots \\ U_{2,1}^{n+1} \\ U_{2,2}^{n+1} \end{bmatrix}$$

For simplification, let $a = (1 + 2r_x + 2r_y)$, $b = (1 - 2r_x - 2r_y)$. Then the system for the right side of the equation $A\mathbf{u}^{n+1} = B\mathbf{u}^n$ is

$$\begin{bmatrix} a & -r_y & 0 & -r_x & 0 & 0 & 0 & 0 & 0 \\ -r_y & a & -r_y & 0 & -r_x & 0 & 0 & 0 & 0 \\ 0 & -r_y & a & 0 & 0 & -r_x & 0 & 0 & 0 \\ -r_x & 0 & 0 & a & -r_y & 0 & -r_x & 0 & 0 \\ 0 & -r_x & 0 & -r_y & a & -r_y & 0 & -r_x & 0 \\ 0 & 0 & -r_x & 0 & -r_y & a & 0 & 0 & -r_x \\ 0 & 0 & 0 & -r_x & 0 & 0 & a & -r_y & 0 \\ 0 & 0 & 0 & 0 & -r_x & 0 & -r_y & a & -r_y \\ 0 & 0 & 0 & 0 & 0 & -r_x & 0 & -r_y & a \end{bmatrix} \begin{bmatrix} U_{0,0}^{n+1} \\ U_{0,1}^{n+1} \\ U_{0,2}^{n+1} \\ U_{1,0}^{n+1} \\ U_{1,1}^{n+1} \\ U_{1,2}^{n+1} \\ U_{2,0}^{n+1} \\ U_{2,1}^{n+1} \\ U_{2,2}^{n+1} \end{bmatrix}$$

and the left side of the equation is

$$\begin{bmatrix} b & r_y & 0 & r_x & 0 & 0 & 0 & 0 & 0 \\ r_y & b & r_y & 0 & r_x & 0 & 0 & 0 & 0 \\ 0 & r_y & b & 0 & 0 & r_x & 0 & 0 & 0 \\ r_x & 0 & 0 & b & r_y & 0 & r_x & 0 & 0 \\ 0 & r_x & 0 & r_y & b & r_y & 0 & r_x & 0 \\ 0 & 0 & r_x & 0 & r_y & b & 0 & 0 & r_x \\ 0 & 0 & 0 & r_x & 0 & 0 & b & r_y & 0 \\ 0 & 0 & 0 & 0 & r_x & 0 & r_y & b & r_y \\ 0 & 0 & 0 & 0 & 0 & r_x & 0 & r_y & b \end{bmatrix} \begin{bmatrix} U^n_{0,0} \\ U^n_{0,1} \\ U^n_{0,2} \\ U^n_{1,0} \\ U^n_{1,1} \\ U^n_{1,2} \\ U^n_{2,0} \\ U^n_{2,1} \\ U^n_{2,2} \end{bmatrix}$$

For every time step, we need to solve this system of equations for this 3x3 lattice region.

In general, for an NxN lattice region:

$$A\mathbf{u}^{n+1} = \begin{bmatrix} a & -r_y & 0 & 0 & \cdots & 0 & -r_y & 0 & 0 & \cdots & & 0 \\ -r_y & a & -r_y & 0 & \cdots & & 0 & -r_y & 0 & \cdots & & 0 \\ 0 & -r_y & a & -r_y & & & & & -r_x & & & \\ 0 & 0 & -r_y & \ddots & \ddots & & & & & \ddots & & \\ \vdots & \vdots & & \ddots & & & & & & & & \\ 0 & & & & & & & & & & & \\ -r_y & 0 & & & & & & & & & & \\ 0 & -r_y & & & & & & & & & & \\ 0 & 0 & -r_y & & & & & & & \ddots & & \\ \vdots & \vdots & & \ddots & & & & & \ddots & \ddots & & -r_x \\ 0 & 0 & & & & & & & & & -r_x & a \end{bmatrix} \begin{bmatrix} U^{n+1}_{0,0} \\ U^{n+1}_{0,1} \\ U^{n+1}_{0,2} \\ \\ \vdots \\ \\ \vdots \\ \\ U^{n+1}_{N,N} \end{bmatrix}$$

And similarly for the right side ($B\mathbf{u}^n$):

$$B\mathbf{u}^n = \begin{bmatrix} b & r_y & 0 & 0 & \cdots & 0 & r_y & 0 & 0 & \cdots & & 0 \\ r_y & b & r_y & 0 & \cdots & & 0 & r_y & 0 & \cdots & & 0 \\ 0 & r_y & b & r_y & & & & & r_x & & & \\ 0 & 0 & r_y & \ddots & \ddots & & & & & \ddots & & \\ \vdots & \vdots & & \ddots & & & & & & & & \\ 0 & & & & & & & & & & & \\ r_y & 0 & & & & & & & & & & \\ 0 & r_y & & & & & & & & & & \\ 0 & 0 & r_y & & & & & & & \ddots & & \\ \vdots & \vdots & & \ddots & & & & & \ddots & \ddots & & r_x \\ 0 & 0 & & & & & & & & & r_x & b \end{bmatrix} \begin{bmatrix} U^n_{0,0} \\ U^n_{0,1} \\ U^n_{0,2} \\ \\ \vdots \\ \\ \vdots \\ \\ U^n_{N,N} \end{bmatrix}$$

It is important to point out, however, that the systems above alone do not account for the boundary conditions. If the boundaries are Dirichlet, then there is an additional constant vector $\mathbf{b}$ that must be added to $B\mathbf{u^n}$ consistently for every time step that keeps the temperatures at those boundaries constant.

$$A\mathbf{u}^{n+1} = B\mathbf{u}^n + \mathbf{b}$$

Interesting things about the Crank-Nicolson

There are multiple methods to numerically modeling parital differential equations, and they are separated by two categories: the explicit and implicit schemes. In an *explicit* scheme, we compute new time values of the grid function $U_{i,j}^{n+1}$ explicitly *only* in terms of the previous time values $U_{i,j}^{n}$. These explicit schemes, however, have a disadvantage: we are forced to take very small time steps to ensure stability. The classic example of an explicit scheme is covered in Chapter 6.2 of J. David Logan's *Applied Partial Differential Equations, 3rd Edition* [3], where the $r = \frac{k\alpha}{h^2}$ must follow the stability condition:

$$r = \frac{k\alpha}{h^2} \leq \frac{1}{2}$$

The Crank-Nicolson is classified as an *implicit* scheme because we are not able to solve for new time values explicitly, but rather we obtain a *system of algebraic equations* with the new advantage: they are stable without putting a constraint on the size of the time step. Therefore, the Crank-Nicolson is *unconditionally stable*, meaning any size of the time step will produce a stable result.

Conclusion

In MA 303, we learned about numerical methods to approximating solutions to differential equations, like Euler's Method, Improved Euler, and Runge-Kutta. In this project, we took a step further into extending these methods to the realm of partial differenital equations with an additional spatial dimension. The Crank-Nicolson is a popular technique for solving partial differential equations with no clear solutions. A similar application of the Crank-Nicolson can be seen in *Solving the 2D Schrödinger equation using the Crank-Nicolson method (2021)* by Arturo Mena [4], and it is in fact a project that inspired me to pursue this method. I would like to express my gratitude for my professor, Dr. Hanh, for teaching me the content required to tackle a project like this, and I appreciate her for allowing me to choose a topic that's somewhat out of the scope of the class.

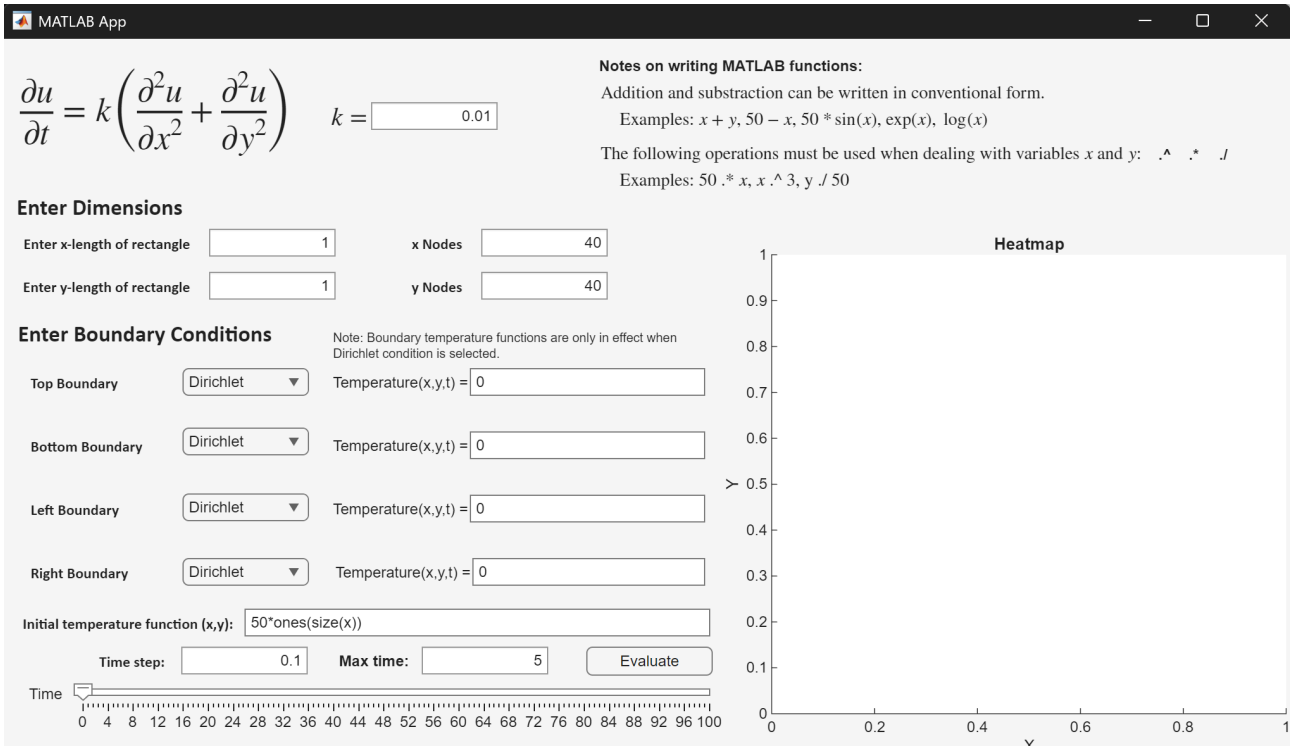


Figure 1

Heat_SIM.mlapp User Manual

I made a MATLAB app using the built-in App Designer to model the heat equation of a rectangular region with customizable paramteres and boundary conditions. I apologetically admit that I have gained assistance from Artificial Intelligence tools to help me set up the code. However, I have done my best to understand the concepts and the methods discussed in this report as much as possible. Therefore, I have not used Artificial Intelligence recklessly, but adequately as a tool to further my understanding.

Step 1: Open the **Heat_SIM.mlapp** while having MATLAB installed in your computer. A window exactly like Figure 1 should pop up.
Step 2: There are general MATLAB syntax instructions written in the upper right corner of the UI window for writing MATLAB functions. Everything else in the window are customizable inputs from the user. Freely change your parameters like k, x length, y length, number of nodes in x and y, time step, and the maximum time you want to run the simulation.

There are boundary conditions on the left of the window which are drop-down menus with two options: Dirichlet and Neumann. When Dirichlet condition is chosen for one of the boundaries, make sure to define the temperature function **Temperature**(x, y, t). This does not need to be done when Neumann condition is chosen.

Step 3: When everything is set, press the **Evaluate** button. The app should display a heatmap that agrees with your initial temperature function. Freely move the Time slider to observe the evolution of the heat content in the region through time.

References

[1] Wikipedia contributors. Tridiagonal matrix algorithm, 4 2026.

[2] Stanley J. Farlow. *Partial Differential Equations for Scientists and Engineers*. Dover Publications, 1 edition, 9 1993.

[3] David Logan. *Applied Partial Differential Equations*. Springer Cham, 3 edition, 12 2014.

[4] Arturo Mena. Solving the 2D Schrödinger equation using the Crank-Nicolson method, 2 2021.